Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

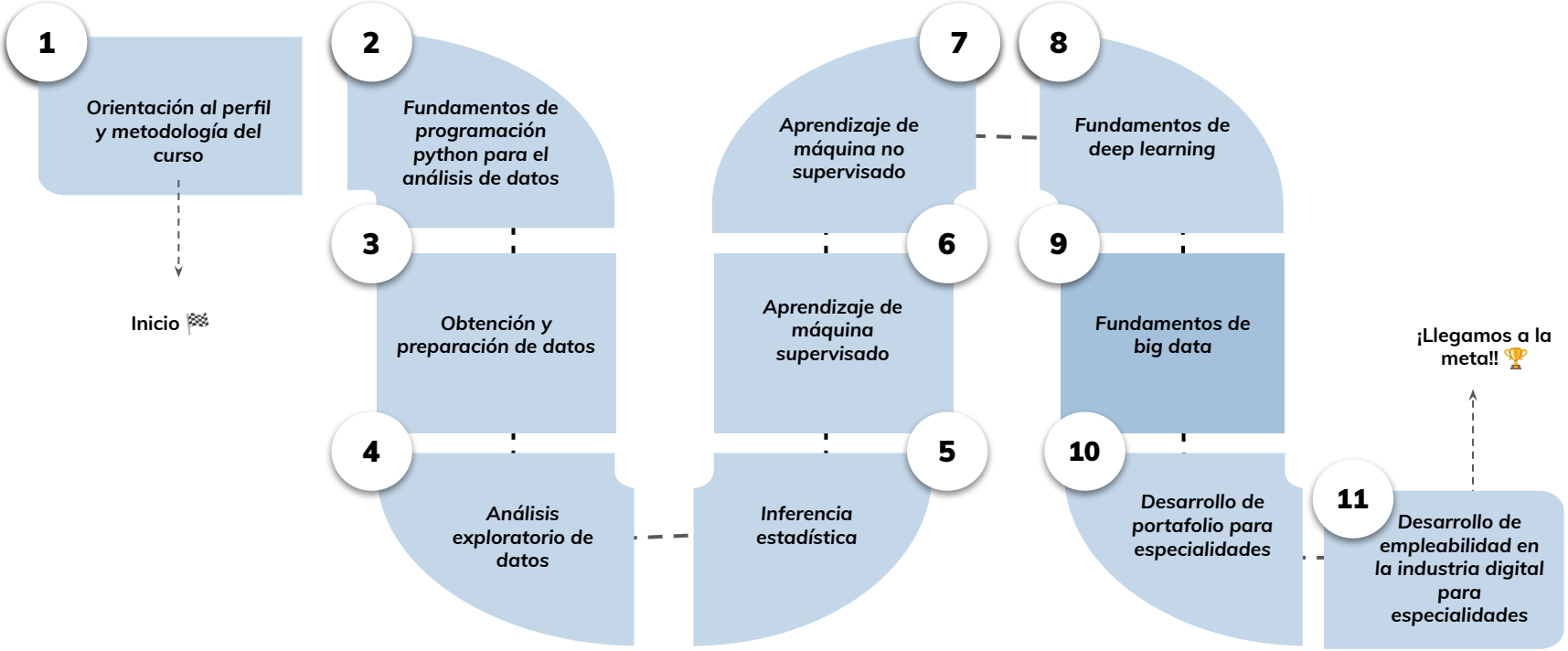
alkemy

› Apache spark- Parte II

Aprendizaje Esperado 2: Explicar las características y componentes principales del framework apache spark para el procesamiento de altos volúmenes de datos.

Hoja de ruta

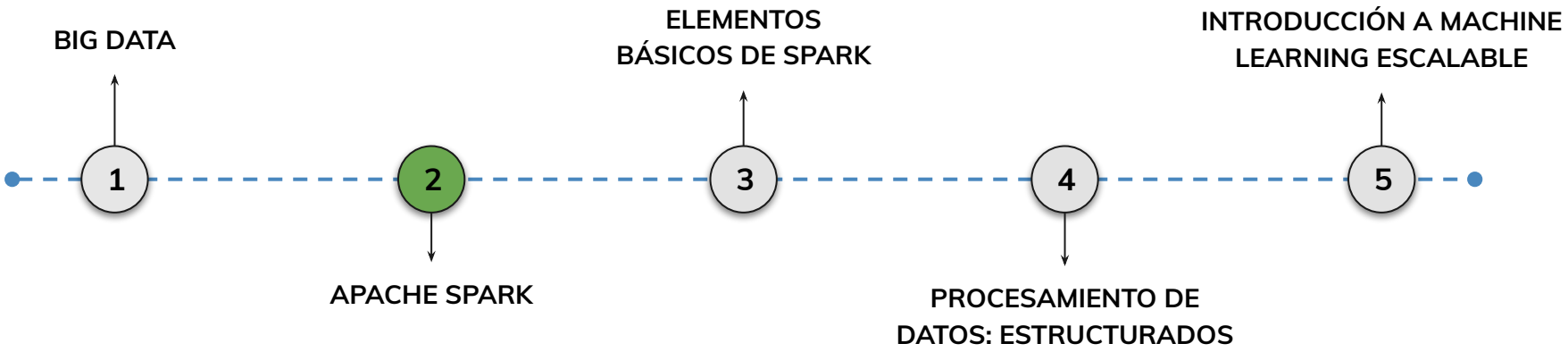
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

2.

Componentes internos y arquitectónicos de Apache Spark

Analizaremos en profundidad la arquitectura de ejecución de Spark, su funcionamiento en modo clúster y sus distintas implementaciones, como PySpark y servicios en la nube.

- **Arquitectura de ejecución distribuida en Spark**
  - **Modelo Maestro-Esclavo.**
  - **Componentes: Driver, Executors, Cluster Manager.**
- **Implementaciones de Spark**
  - **Despliegue en modo clúster.**
  - **Servicios gestionados en la nube (AWS, GCP, Azure).**

Objetivos de aprendizaje

¿Qué aprenderás?

- Describir la arquitectura interna de Apache Spark y cómo se distribuyen las tareas.
- Explicar el rol del Driver, Executors y el Cluster Manager.
- Analizar cómo Spark divide y ejecuta Jobs en Stages y Tasks.
- Comprender el funcionamiento de Spark en modo clúster.
- Identificar distintas formas de implementar Spark (nube, PySpark, Kubernetes).

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Comprendimos qué es Apache Spark y cómo difiere de otros frameworks como Hadoop MapReduce.*

→ *Exploramos su arquitectura general: SparkContext, Driver, Executors y Cluster Manager.*

→ *Investigamos sus componentes clave: RDD, DataFrames y Datasets.*

→ *Vimos los distintos módulos del ecosistema Spark: SQL, Streaming, MLlib y GraphX.*

Apache spark

Conclusión sobre cuándo usar Spark

En conclusión, Spark conviene ser utilizado en contextos donde se requiere **alto rendimiento, escalabilidad, análisis complejos y procesamiento en tiempo real**. La elección adecuada dependerá siempre de los objetivos del proyecto, el volumen de datos y los recursos disponibles.

API de Spark: Flexibilidad y facilidad de uso

Uno de los aspectos que ha contribuido al éxito y adopción masiva de **Apache Spark** es la flexibilidad y facilidad de uso de su **API** (Interfaz de Programación de Aplicaciones). Spark ofrece un conjunto de APIs que permiten a los desarrolladores y analistas de datos interactuar con el motor de procesamiento de manera sencilla y eficiente.

Spark soporta varios lenguajes de programación, incluyendo **Scala**, **Java**, **Python** y **R**, lo que facilita su integración en diferentes entornos y equipos de trabajo. Esta flexibilidad permite que tanto desarrolladores como científicos de datos puedan aprovechar las capacidades de Spark sin necesidad de aprender un nuevo lenguaje.

API de Spark: Estructuras de datos distribuidas

RDD (Resilient Distributed Dataset)

El núcleo de la API de Spark se basa en la manipulación de estructuras de datos distribuidas. La primera y más fundamental de estas estructuras es el **RDD (Resilient Distributed Dataset)**, una colección inmutable y distribuida de objetos que pueden ser procesados en paralelo. Los RDDs permiten aplicar operaciones de transformación (como map, filter) y acción (como count, collect) sobre los datos.

DataFrame

Los **DataFrames** son colecciones distribuidas de datos organizados en columnas, similares a una tabla en una base de datos relacional. Esta estructura ofrece mayor eficiencia y permite realizar consultas SQL sobre los datos.

Dataset

Los **Datasets** combinan la eficiencia de los DataFrames con el tipado fuerte de los RDDs, ofreciendo un balance entre seguridad y flexibilidad.

API de Spark: Spark SQL

Spark también proporciona una API para realizar consultas estructuradas mediante **Spark SQL**. Esta interfaz permite ejecutar consultas SQL estándar sobre DataFrames y Datasets, facilitando el trabajo a los usuarios que ya tienen experiencia en lenguaje SQL.

API de Spark: Spark Streaming

Para el procesamiento en tiempo real, Spark ofrece la API de **Spark Streaming**, que permite procesar flujos de datos en micro-lotes. Esta API es compatible con DataFrames y Datasets, y soporta múltiples fuentes de datos en tiempo real, como Kafka, Flume o sockets TCP.

API de Spark: MLlib y GraphX

MLlib

En el ámbito del aprendizaje automático, Spark incluye la biblioteca **MLlib**, que proporciona algoritmos y herramientas para clasificación, regresión, clustering, reducción de dimensionalidad y evaluación de modelos. La API de MLlib permite construir y entrenar modelos sobre grandes volúmenes de datos de manera eficiente.

GraphX

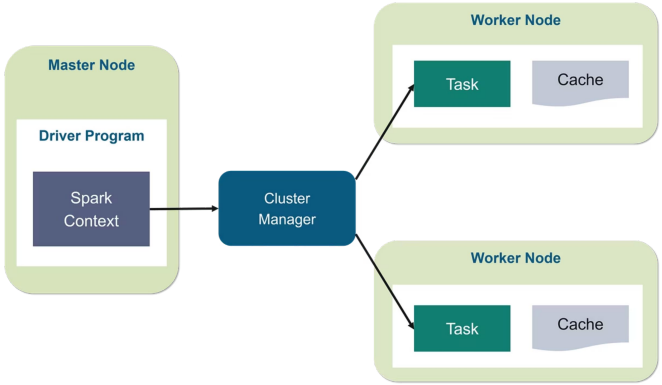
Además, Spark ofrece la API de **GraphX** para el procesamiento y análisis de grafos. Esta API permite modelar y manipular grafos distribuidos, facilitando tareas como la búsqueda de rutas, detección de comunidades y análisis de conectividad.

Resumen de la API de Spark

En resumen, la API de Spark es uno de sus puntos más fuertes, ya que proporciona un entorno unificado para realizar procesamiento por lotes, consultas SQL, análisis en tiempo real, aprendizaje automático y análisis de grafos, todo bajo un modelo sencillo y flexible.

Arquitectura general de Spark

La arquitectura de **Apache Spark** está diseñada para aprovechar el procesamiento distribuido y en memoria, permitiendo manejar grandes volúmenes de datos de manera eficiente y escalable. Comprender su arquitectura es esencial para aprovechar al máximo las capacidades que ofrece este framework.



Modelo Maestro-Esclavo de Spark

Modelo Maestro-Esclavo

La arquitectura de Spark se basa en un modelo **Maestro-Esclavo (Master-Slave)**, en el que existe un nodo principal encargado de la coordinación y varios nodos trabajadores encargados del procesamiento de datos. Este modelo permite distribuir las tareas de procesamiento y garantizar la tolerancia a fallos.

Driver Program

En el centro de la arquitectura se encuentra el **Driver Program**, que actúa como punto de entrada de cualquier aplicación Spark. El Driver es responsable de convertir el código escrito por el usuario en tareas distribuidas y enviarlas a los nodos del clúster para su ejecución. Además, mantiene el estado de la aplicación y coordina la distribución y ejecución de las tareas.

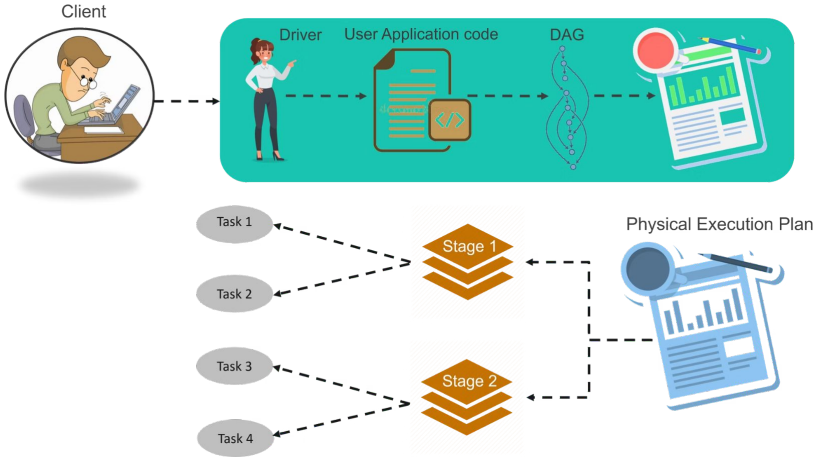
Cluster Manager

El Driver se comunica con un componente llamado **Cluster Manager**, encargado de gestionar los recursos del clúster. Existen diferentes opciones de Cluster Manager que pueden ser utilizadas con Spark, como **Standalone**, **Apache Mesos**, **Hadoop YARN** o gestores de clúster en la nube como Kubernetes.

Executors y RDDs en la arquitectura de Spark

Los nodos trabajadores del clúster son conocidos como **Executors**. Cada Executor es un proceso que ejecuta las tareas asignadas por el Driver y almacena en memoria los datos intermedios. Los Executors también son responsables de reportar el progreso de las tareas al Driver.

Una de las características clave de la arquitectura de Spark es el uso de **RDDs (Resilient Distributed Datasets)**, que permiten dividir los datos en particiones distribuidas entre los diferentes Executors. Este enfoque facilita el procesamiento paralelo y garantiza la recuperación de los datos en caso de fallos.



Jobs y Stages en la arquitectura de Spark

Código de usuario

El usuario escribe código que define transformaciones y acciones sobre los datos.

Job

Spark organiza el procesamiento en **Jobs** y **Stages**. Un **Job** es una acción solicitada por el usuario, como contar los registros de un RDD.

Stage

Cada Job está compuesto por uno o más **Stages**. Cada Stage consiste en un conjunto de tareas que pueden ejecutarse en paralelo sobre las particiones de los datos.

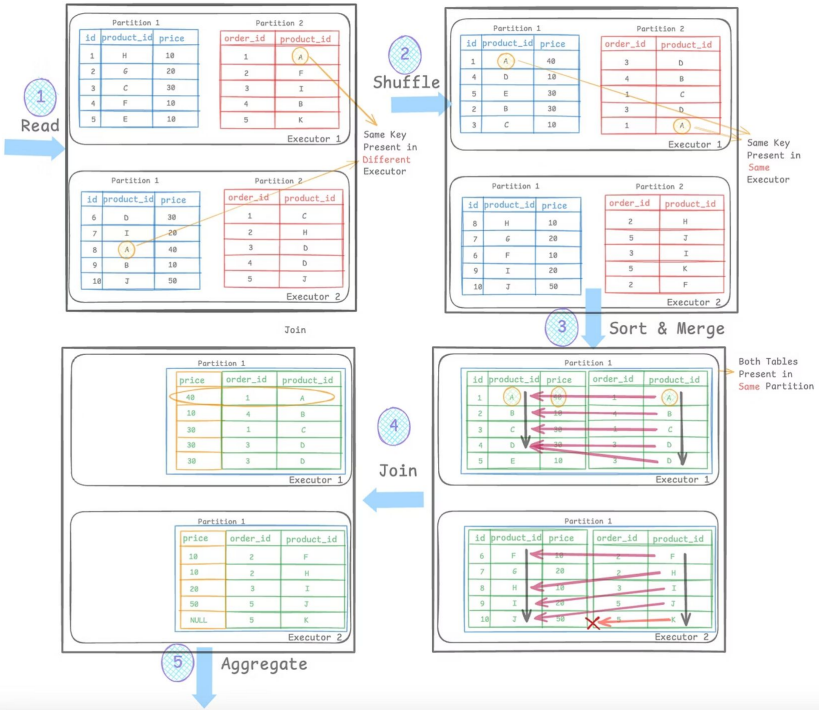
Task

Las tareas son enviadas a los Executors para su ejecución en paralelo.

Flujo de ejecución y tolerancia a fallos

El flujo general de ejecución en Spark comienza cuando el usuario envía una acción. El Driver genera un Job y lo divide en Stages. Cada Stage se divide en tareas que son enviadas a los Executors para su ejecución. Los resultados son recolectados y enviados de vuelta al Driver.

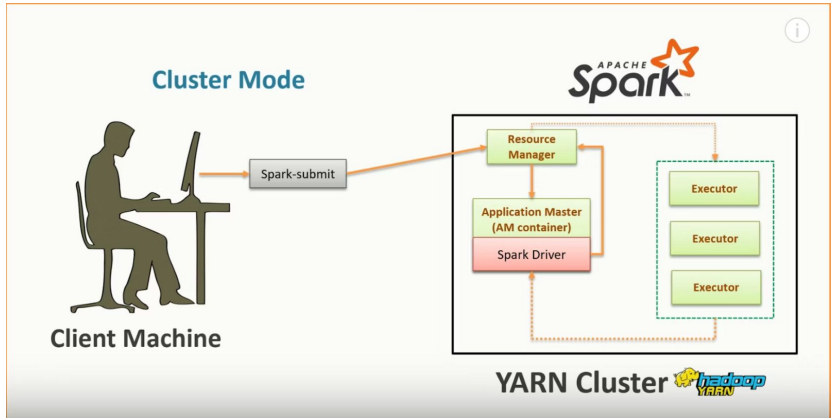
La arquitectura de Spark también incluye mecanismos de **tolerancia a fallos**. Si un Executor falla, las tareas asignadas pueden ser reasignadas a otro Executor utilizando la información del lineage de los RDDs.



Principales componentes de Spark: Driver

La arquitectura de **Apache Spark** está formada por varios componentes esenciales que trabajan juntos para garantizar el procesamiento eficiente y distribuido de los datos. Los tres elementos clave de esta arquitectura son el **Driver**, los **Executors** y el **Cluster Manager**.

El **Driver** es el componente principal que inicia y gestiona la ejecución de una aplicación Spark. Se encarga de crear el **SparkContext**, que es el punto de entrada para cualquier aplicación Spark, y de coordinar la ejecución de las tareas en el clúster. El Driver también realiza el seguimiento del estado de la aplicación y recopila los resultados obtenidos por los Executors.



Principales componentes de Spark: DAG y Stages

Además, el Driver transforma las operaciones solicitadas por el usuario en un **Directed Acyclic Graph (DAG)**, que representa las dependencias entre las tareas y permite optimizar el plan de ejecución. Este DAG es dividido en etapas (Stages) que serán distribuidas y ejecutadas por los nodos trabajadores.

Principales componentes de Spark: Executors

Definición de Executors

Los **Executors** son procesos que se ejecutan en los nodos del clúster y son responsables de realizar las tareas asignadas por el Driver. Cada Executor procesa una partición de los datos, almacena en memoria los datos intermedios y devuelve los resultados al Driver una vez completadas las tareas. Los Executors también informan al Driver sobre el estado y el progreso de las tareas.

Configuración de Executors

La eficiencia de Spark depende en gran medida de la correcta configuración y utilización de los Executors. Es posible ajustar el número de Executors, la cantidad de memoria asignada y la cantidad de núcleos utilizados para optimizar el rendimiento de las aplicaciones.

Principales componentes de Spark: Cluster Manager

El tercer componente clave es el **Cluster Manager**, que gestiona los recursos físicos del clúster y asigna los recursos necesarios a las aplicaciones Spark. El Cluster Manager se encarga de iniciar y monitorear los Executors, asegurando que las tareas se distribuyan equitativamente entre los nodos.

Standalone

Es el gestor de recursos nativo de Spark, simple y fácil de configurar.

Apache Mesos

Un gestor de clústeres general que permite compartir recursos entre diferentes aplicaciones distribuidas.

Hadoop YARN

Un gestor de recursos diseñado para entornos Hadoop, que permite ejecutar aplicaciones Spark junto con otras aplicaciones Hadoop.

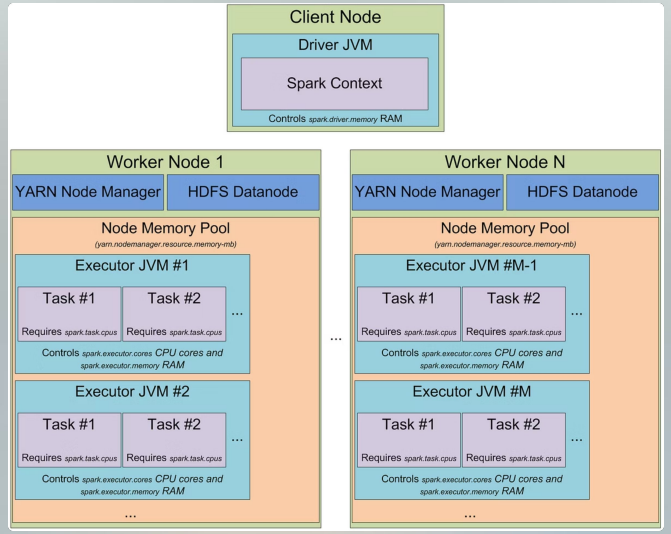
Kubernetes

Un sistema de orquestación de contenedores que permite desplegar y gestionar aplicaciones Spark en entornos basados en contenedores.

Interacción entre componentes de Spark

La interacción entre el Driver, los Executors y el Cluster Manager permite a Spark distribuir el procesamiento de los datos, garantizar la tolerancia a fallos y escalar horizontalmente para gestionar volúmenes de datos masivos.

En conclusión, comprender el rol y funcionamiento de estos tres componentes es fundamental para diseñar, configurar y ejecutar aplicaciones Spark de manera eficiente y efectiva.

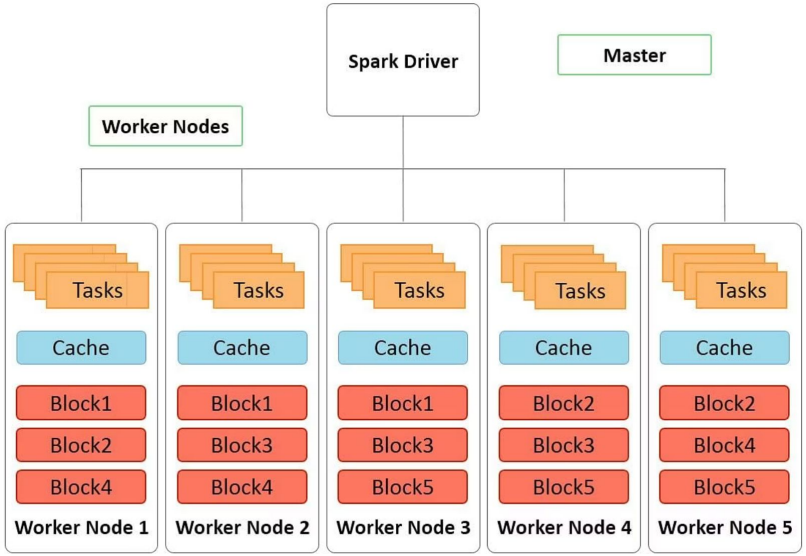


Modo de funcionamiento en clúster

El verdadero potencial de **Apache Spark** se despliega cuando se ejecuta en un entorno distribuido, conocido como **modo clúster**. Este modo permite procesar grandes volúmenes de datos utilizando múltiples nodos, aprovechando la potencia de cómputo y la capacidad de almacenamiento de todos los servidores que forman parte del clúster.

Distribución de aplicaciones en modo clúster

Cuando Spark se ejecuta en modo clúster, la aplicación se distribuye entre un nodo maestro y varios nodos trabajadores. El nodo maestro aloja el **Driver Program**, que coordina la ejecución de la aplicación, mientras que los nodos trabajadores ejecutan los **Executors**, encargados de procesar los datos y realizar las tareas asignadas.



Opciones de despliegue en modo clúster

1

Spark Standalone

El modo más sencillo, que utiliza un gestor de clúster propio de Spark.

- Fácil configuración y despliegue
- Ideal para entornos dedicados a Spark
- Menor flexibilidad para compartir recursos

2

Hadoop YARN

Integra Spark con un ecosistema Hadoop existente, permitiendo compartir recursos con otras aplicaciones.

- Buena integración con HDFS
- Gestión eficiente de recursos
- Soporte para múltiples aplicaciones

3

Apache Mesos

Un gestor de clúster más general, que soporta múltiples tipos de aplicaciones distribuidas.

- Alta flexibilidad
- Soporte para contenedores
- Escalabilidad avanzada

4

Kubernetes

Permite desplegar Spark en entornos de contenedores, facilitando la orquestación y escalabilidad.

- Gestión basada en contenedores
- Despliegue automatizado
- Integración con CI/CD

Flujo de trabajo en modo clúster

Envío de la aplicación

El flujo de trabajo en modo clúster comienza cuando un usuario envía una aplicación Spark al Cluster Manager.

División de tareas

El Driver divide las operaciones solicitadas en tareas y las distribuye a los Executors para su ejecución.

Asignación de recursos

El Cluster Manager asigna los recursos necesarios y lanza los Executors en los nodos trabajadores.

Ejecución y recolección

Los Executors procesan las tareas y devuelven los resultados al Driver.

Ventajas del modo clúster

Escalabilidad horizontal

Una de las ventajas del modo clúster es la **escalabilidad horizontal**. Es posible añadir más nodos al clúster para incrementar la capacidad de procesamiento y almacenamiento de datos, adaptándose al crecimiento de la demanda.

Tolerancia a fallos

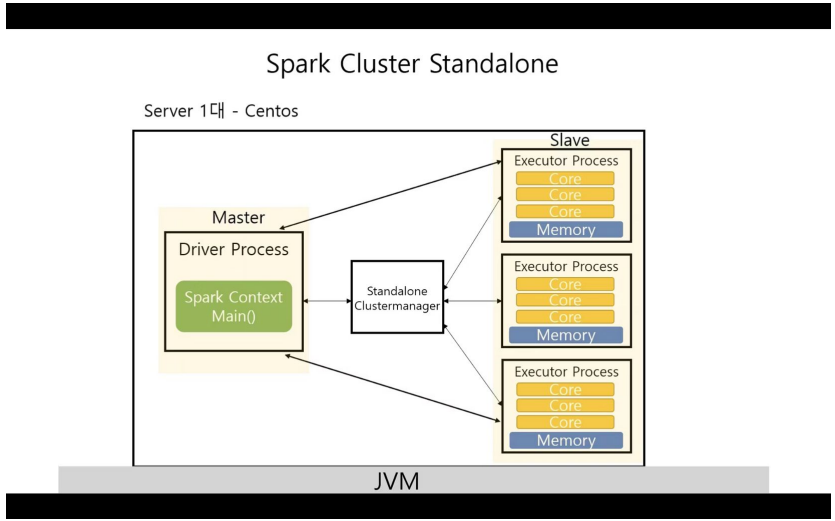
Además, el modo clúster proporciona **tolerancia a fallos**. Si un nodo falla durante la ejecución, el Cluster Manager puede reasignar las tareas a otros nodos disponibles, garantizando la continuidad del procesamiento.

Optimización de recursos

El modo clúster también permite configurar diferentes políticas de asignación de recursos y balanceo de carga, optimizando el uso de la infraestructura disponible y asegurando que ninguna aplicación consuma más recursos de los necesarios.

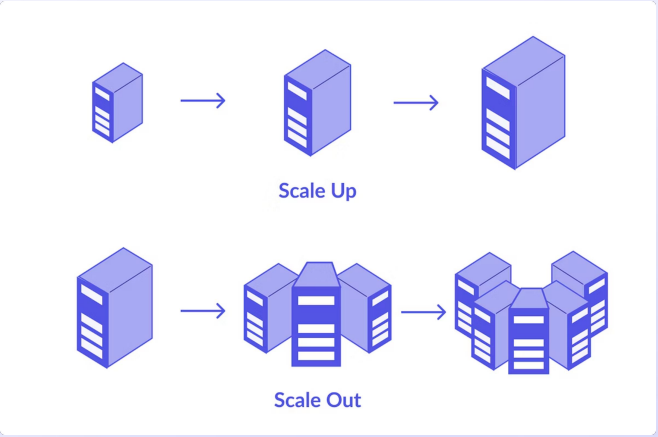
Consideraciones para el modo clúster

Sin embargo, la ejecución de Spark en modo clúster requiere una correcta planificación y configuración. Aspectos como la asignación de memoria y núcleos, la ubicación de los datos y la seguridad deben ser considerados para garantizar un rendimiento óptimo y la protección de la información.



Conclusión sobre el modo clúster

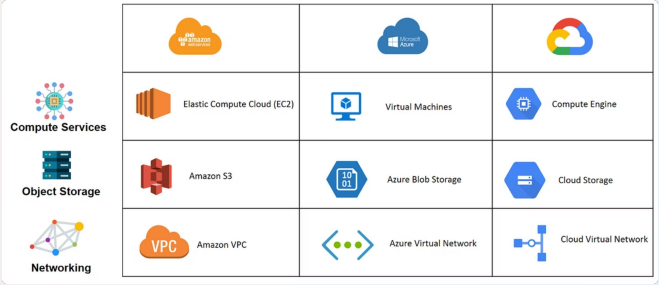
En conclusión, el modo de funcionamiento en clúster es fundamental para aprovechar la verdadera potencia de Spark, permitiendo procesar grandes volúmenes de datos de manera distribuida, eficiente y escalable.



Implementaciones de Spark: Spark en la nube

El ecosistema de **Apache Spark** ha evolucionado para adaptarse a diferentes entornos y necesidades, dando lugar a diversas implementaciones que facilitan su adopción y uso en distintos contextos. Entre las más destacadas se encuentran **Spark en la nube** y **PySpark**.

Una de las implementaciones más utilizadas es **Spark en la nube**. Las principales plataformas de computación en la nube, como **Amazon Web Services (AWS)**, **Google Cloud Platform (GCP)** y **Microsoft Azure**, ofrecen servicios gestionados de Apache Spark, eliminando la necesidad de que las organizaciones configuren y administren la infraestructura subyacente.



Servicios de Spark en la nube

Amazon EMR

En **AWS**, el servicio **Amazon EMR (Elastic MapReduce)** permite desplegar clústeres Spark de forma rápida y escalable, integrándolos fácilmente con otros servicios de almacenamiento y análisis de datos de la nube.

Google Dataproc

En **Google Cloud**, **Dataproc** proporciona un servicio similar, facilitando la ejecución de Spark con integración nativa en el ecosistema Google.

Azure HDInsight

Por su parte, **Azure HDInsight** ofrece un entorno optimizado para Spark con integración en los servicios de Microsoft.

Ventajas de Spark en la nube

Escalabilidad dinámica

Permite a las organizaciones **escalar dinámicamente** los recursos según la demanda, ajustando la capacidad de procesamiento en tiempo real.

Modelo de pago por uso

Las organizaciones pueden pagar únicamente por lo que consumen, reduciendo los costos de infraestructura.

Integración con servicios

Facilita la integración con servicios de almacenamiento, bases de datos y análisis avanzados disponibles en la nube.

Reducción de complejidad

Reduce la complejidad operativa al eliminar la necesidad de gestionar la infraestructura subyacente.

Implementaciones de Spark: PySpark

Otra implementación clave es **PySpark**, la interfaz de Apache Spark para el lenguaje de programación **Python**. PySpark permite a los desarrolladores y analistas de datos escribir aplicaciones Spark utilizando Python, uno de los lenguajes más populares en el ámbito de la ciencia de datos.

Características de PySpark

Acceso completo a Spark

PySpark ofrece acceso a todas las funcionalidades de Spark, incluyendo la creación y manipulación de **RDDs**, **DataFrames** y **Datasets**, así como el uso de módulos avanzados como **Spark SQL**, **Spark Streaming**, **MLlib** y **GraphX**. Esto facilita la integración de Spark en flujos de trabajo de análisis de datos y aprendizaje automático basados en Python.

Integración con ecosistema Python

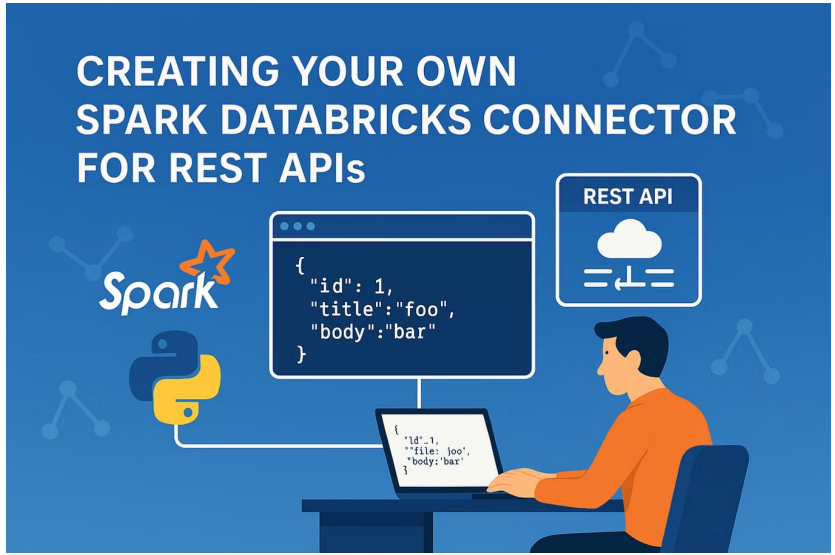
Uno de los beneficios de PySpark es que permite combinar la potencia del procesamiento distribuido de Spark con la riqueza del ecosistema Python, incluyendo librerías como **Pandas**, **NumPy** y **Scikit-learn**. Esto resulta especialmente útil para los equipos de ciencia de datos que ya trabajan con estas herramientas.

Desarrollo y pruebas simplificados

Además, PySpark simplifica el desarrollo y la prueba de aplicaciones Spark, ya que permite ejecutar código localmente antes de escalarlo a un clúster distribuido.

Consideraciones sobre PySpark

Sin embargo, es importante tener en cuenta que, al ejecutarse sobre la **Java Virtual Machine (JVM)**, PySpark puede presentar cierta sobrecarga y menor rendimiento en comparación con aplicaciones escritas directamente en Scala o Java. A pesar de ello, su facilidad de uso y compatibilidad con el ecosistema Python lo convierten en una opción muy popular.



Ejemplo de código PySpark

```
from pyspark.sql import SparkSession# Iniciar una sesión de Sparkspark = SparkSession.builder \ .appName("Ejemplo PySpark") \ .getOrCreate()# Crear un DataFrame a partir de un archivo CSVdf = spark.read.csv("datos.csv", header=True, inferSchema=True)# Realizar transformacionesresultado = df.filter(df.edad > 25) \ .groupBy("departamento") \ .count()# Mostrar resultadosresultado.show()# Cerrar la sesiónspark.stop()
```

Otras implementaciones de Spark

Spark en Kubernetes

Permite desplegar aplicaciones Spark en entornos de contenedores orquestados por Kubernetes, facilitando la portabilidad y la gestión de recursos.

Spark en Edge Computing

Implementaciones adaptadas para ejecutar Spark en dispositivos de borde, permitiendo el procesamiento de datos cerca de donde se generan.

Spark con GPU

Versiones optimizadas para aprovechar la potencia de procesamiento de las unidades de procesamiento gráfico, especialmente útiles para tareas de aprendizaje profundo.

Conclusión

A lo largo de este manual, hemos explorado los aspectos fundamentales de **Apache Spark**, una de las herramientas más potentes y utilizadas en el ecosistema Big Data. Comprendimos su origen y la necesidad que impulsó su desarrollo como respuesta a las limitaciones de tecnologías anteriores como Hadoop MapReduce.

Abordamos los conceptos básicos de Spark, incluyendo su modelo de programación, estructuras de datos y componentes esenciales. También analizamos los problemas que Spark resuelve y lo comparamos con otros sistemas de procesamiento, identificando los escenarios ideales para su implementación.

Además, estudiamos en detalle la arquitectura general de Spark, sus principales componentes y su modo de funcionamiento en clúster, aspectos clave para entender su escalabilidad y eficiencia. Finalmente, revisamos sus implementaciones más relevantes, como **Spark en la nube** y **PySpark**, que han facilitado su adopción en múltiples contextos.
El conocimiento adquirido en este manual proporciona una base sólida para que los estudiantes y profesionales puedan desarrollar, desplegar y gestionar soluciones basadas en Spark, aprovechando su capacidad para procesar datos masivos de manera rápida, flexible y escalable.

Live Coding

¿En qué consistirá la Demo?

Esta vez no haremos una codificación real, sino que abordaremos el funcionamiento de Spark desde una perspectiva técnica aplicada, investigando y esquematizando su ejecución.

1. *¿Cómo se transforma una operación en Spark en un DAG y luego en Stages y Tasks?*
2. *¿Qué sucede en la arquitectura cuando una aplicación Spark se ejecuta en modo clúster?*
3. *¿Cómo se comunican el Driver y los Executors? ¿Qué pasa ante un fallo?*
4. *¿Cuáles son las diferencias entre usar Spark en Standalone vs YARN vs Kubernetes?*
5. *¿Qué beneficios concretos ofrece PySpark frente a otros lenguajes?*

Tiempo: 25 Minutos

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio

Mapea el flujo interno de Spark

Mapea el flujo interno de Spark

Contexto: 🙌

Te convertirás en arquitecto/a de datos y deberás comprender cómo Spark distribuye el trabajo internamente y en qué escenarios conviene usar ciertas implementaciones.

Consigna: ✍️

1. Dibuja o esquematiza el flujo de ejecución de Spark desde que se lanza una acción hasta que se completan las tareas.
2. Describe el rol de cada componente: Driver, Executors, Cluster Manager.
3. Elige una implementación de Spark (AWS EMR, PySpark, Kubernetes, etc.) y explica por qué y cuándo conviene usarla.
4. Enumera tres buenas prácticas para correr Spark en modo clúster de forma eficiente.

Paso a paso: ⚙️

1. Crea un gráfico simple (puede ser en papel o digital).
2. Escribe una breve justificación para tu implementación elegida.
3. Prepara un mini-pitch de 2 minutos para presentarlo en clase.

Tiempo 🕐**:** *45 Minutos*

alkemy

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Analizamos en detalle la arquitectura distribuida de Apache Spark.***
- ✓ ***Comprendimos el funcionamiento del Driver, los Executors y el Cluster Manager.***
- ✓ ***Explicamos cómo Spark ejecuta Jobs mediante DAGs, Stages y Tasks.***
- ✓ ***Exploramos el modo clúster, incluyendo tolerancia a fallos y escalabilidad horizontal.***
- ✓ ***Conocimos implementaciones prácticas: Spark en la nube, PySpark, Kubernetes.***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy